



Actividad

Caso 3-A: Logística de Distribución Regional

Nombre de la escuela

Universidad Politécnica de Chiapas

Autores

243723 Calderón Trejo María Regina
243737 Morales Montejo Ian Mauricio
243706 Selvas Alvarez Omar Kalid
5B

Fecha

05 de Febrero 2026

Lugar

Suchiapa, Chiapas, México

Materia

Base de Datos Avanzadas

Profesor

Ramsés Alejandro Camas Nájera

Restricciones identificadas

- RFC (fk) único
- price > 0
- quantity > 0
- Placas de vehículo únicas (no se puede repetir).
- Licencia de conductor única (no se puede repetir).
- Cantidad en pedido > 0.
- descriptions NOT NULL
- NOT NULL: En todos los campos de IDs y nombres.
- CHECK (stock >= 0): En la tabla Products.
- CHECK (status): En Vehicles, limitado a ('Disponible', 'Mantenimiento', 'En ruta').
- CHECK (type): En inventory_movement, limitado a ('Entrada', 'Salida').
- FK Integridad: ON DELETE RESTRICT en productos que ya tienen movimientos registrados.
- DEFAULT: date_creation como CURRENT_TIMESTAMP en pedidos e incidencias.
- CHECK (capacity > 0): En la tabla Vehicles.

Querys sql de ejemplo

1. Productos activos con artesano y comunidad (JOINS). Obtenemos las listas de productos ligada a sus proveedores y categorías.

```
1 SELECT
2     p.name AS producto,
3     s.name AS proveedor,
4     c.name AS categoria
5 FROM Products p
6 JOIN Suppliers s ON p.suppliers_id = s.id [cite: 332]
7 JOIN Categories c ON p.categories_id = c.id; [cite: 331]
```

2. Peso total por ruta vs capacidad. Comparamos la carga asignada contra el límite del vehículo para evitar sobrecargas.

```
1 SELECT
2     r.id AS ruta_id,
3     v.plates AS placas,
4     v.capacity AS capacidad_max,
5     SUM(p.stock) AS peso_estimado_carga
6 FROM routes r
7 JOIN Vehicles v ON r.vehicles_plates = v.plates [cite: 407]
8 JOIN route_assignments ra ON r.id = ra.routes_id [cite: 398]
9 JOIN order_details od ON ra.order_id = od.id
10 JOIN Products p ON od.products_id = p.id [cite: 358]
11 GROUP BY r.id, v.plates, v.capacity;
```

3. Comparadores. que más han gastado (GROUP BY + HAVING).
Identificamos los clientes más importantes que han superado los \$10,000 MXN en compras.

```
1 SELECT
2     b.name AS empresa,
3     COUNT(o.id) AS num_pedidos,
4     SUM(od.quantity * p.sale_price) AS total_gastado [cite: 358, 336]
5 FROM Business b
6 JOIN orders o ON b.id = o.business_id [cite: 339]
7 JOIN order_details od ON o.id = od.id
8 JOIN Products p ON od.products_id = p.id
9 GROUP BY b.id, b.name
10 HAVING SUM(od.quantity * p.sale_price) > 10000; [cite: 11]
```

4. Historial de cambios de estado. Muestra el flujo de movimientos de inventario para auditorías.

```
1  SELECT
2      p.name AS producto,
3      im.type AS tipo_movimiento,
4      im.quantity,
5      im.description
6  FROM inventory_movement im
7  JOIN Products p ON im.products_id = p.id [cite: 346]
8  ORDER BY im.id DESC;
```

5. Uso de CASE y COALESCE para incidencias. Muestra los pedidos y, si no tiene descripción de incidencia la asigna por defecto.

```
1  SELECT
2      o.id AS pedido_id,
3      COALESCE(oi.description, 'Sin incidencias reportadas')
4      AS reporte_estado, [cite: 393]
5      CASE
6          WHEN oi.id IS NOT NULL THEN 'Revisión Requerida'
7          ELSE 'OK'
8      END AS semaforo_logistico
9  FROM orders o
10 LEFT JOIN orders_incidents oi ON o.id = oi.orders_id; [cite: 389]
```

Views

Vista 1: Resumen de pedidos por zona geográfica Muestra la efectividad de las entregas en Chiapas, Oaxaca y Tabasco

```
CREATE VIEW vista_cobertura_zona AS
SELECT
    b.zone,
    COUNT(o.id) AS total_pedidos,
    COUNT(oi.id) AS pedidos_con_incidencia
FROM Businnes b
JOIN orders o ON b.id = o.ibunnises_id
LEFT JOIN orders_incidents oi ON o.id = oi.orders_id
GROUP BY b.zone;
```

Vista 2: Productividad de conductores Clasifica el rendimiento de los conductores según las rutas completadas.

```
CREATE VIEW vista_productividad_conductores AS
SELECT
    d.name AS conductor,
    COUNT(r.id) AS rutas_realizadas,
    CASE
        WHEN COUNT(r.id) > 10 THEN 'Alto Rendimiento'
        WHEN COUNT(r.id) BETWEEN 5 AND 10 THEN
'Estándar'
        ELSE 'Baja Actividad'
    END AS clasificacion
FROM Drivers d
JOIN routes r ON d.license = r.drivers_license
GROUP BY d.license, d.name;
```

Vista 3: Inventario Crítico: Calcula si el stock dura menos de 15 días, manejando productos sin ventas recientes.

```

CREATE VIEW vista_inventario_critico AS
SELECT
    p.name,
    p.stock,
    CASE
        WHEN p.stock = 0 THEN 'AGOTADO'
        WHEN p.stock < 20 THEN 'REABASTECER URGENTE'
        ELSE 'Suficiente'
    END AS estatus
FROM Products p;

```

Vista 4: Rentabilidad por Proveedor: Compara lo gastado en compra y lo obtenido en venta.

```

CREATE VIEW vista_rentabilidad_proveedores AS
SELECT
    s.name AS proveedor,
    SUM(p.stock * p.purchase_price) AS inversion_total,
    SUM(p.stock * p.sale_price) AS venta_potencial,
    SUM(p.stock * (p.sale_price - p.purchase_price)) AS margen_ganancia
FROM Suppliers s
JOIN Products p ON s.id = p.suppliers_id
GROUP BY s.id, s.name;

```

Vista 5: Utilización de Flota Muestra el estado de los vehículos y cuántas rutas han operado en el mes

```

CREATE VIEW vista_utilizacion_flota AS
SELECT
    v.plates,
    v.status,
    COUNT(r.id) AS total_rutas_mes
FROM Vehicles v
LEFT JOIN routes r ON v.plates = r.vehicles_plates
GROUP BY v.plates, v.status;

```

Preguntas

¿Por qué escogieron ese caso?

Llegamos a la conclusión de escoger este caso, ya que sentimos que estamos más familiarizados con los casos tipo e-commerce debido a previos proyectos en los cuales los modelos eran de forma similar.

Tenemos una mejor noción de cómo organizar las entidades de las tablas, así como las relaciones entre estas y así poder encontrar una solución eficiente a la problemática planteada.

¿Por qué lo diseñaron así?

El diseño que se propuso sigue los principios de normalización, cada entidad tiene sus atributos correspondientes de manera lógica. Decidimos separar a los proveedores (Suppliers), categorías (Categories), zonas (Zones), vehículos (Vehicles) y conductores (Drivers) como entidades independientes. Esto evita la redundancia: si un proveedor cambia su RFC o un conductor actualiza su teléfono, solo lo editamos en un lugar y la actualización se refleja en todo el sistema sin generar errores de consistencia.

Implementamos dos tablas puente fundamentales para las relaciones N:N.

- order_details: Es vital porque un pedido puede llevar múltiples productos de distintos proveedores (como el caso de Don Pepe que pidió leche y maíz). Aquí agregamos el precio histórico; si el precio del queso sube mañana, los pedidos de hoy mantienen el costo al que se vendieron originalmente, protegiendo la integridad financiera.
- route_assignments: Diseñamos esta tabla para que el coordinador logístico tenga flexibilidad. Un vehículo puede llevar muchos pedidos en una sola ruta, y esto nos permite calcular si el peso total asignado no supera la capacidad de la camioneta

No usamos solo un campo de 'stock' en la tabla de productos. Creamos inventory_movement para registrar cada entrada (de proveedores) y salida (hacia pedidos). Esto permite generar el reporte de 'Inventario Crítico', calculando la velocidad de ventas incluso para productos que no han tenido movimientos recientes.

Diseñamos la tabla orders_incidents como una entidad dependiente del pedido. Cuando Carlos Hernández entregó el queso dañado en Juchitán, el sistema permite

registrar esa 'entrega parcial' sin borrar el pedido original, manteniendo la evidencia para posibles reclamos al proveedor o al seguro.

Usamos llaves foráneas y restricciones CHECK para blindar el negocio. Por ejemplo, si un vehículo está en 'mantenimiento', el sistema bloquea su asignación a una ruta.